

Linguaggi di Programmazione (Analisi Lessicale e Sintattica)



Sono le prime due fasi di un compilatore

Lucidi basati su materiale
in inglese di Ras Bodik
<http://inst.eecs.berkeley.edu/~cs164/fa04>

Three kinds of execution environments

- Traduzione a Run-Time
- **Interpreters**

L'interprete esegue il codice sorgente riga per riga, traducendo ed eseguendo immediatamente ogni istruzione. Non produce un file eseguibile separato.

- Scheme, lisp, perl
- popular interpreted languages later got compilers

Traduzione prima del Run-Time

- **Compilers**

Il compilatore traduce l'intero codice sorgente (scritto in un linguaggio ad alto livello) in un codice macchina (o in un codice intermedio, come il bytecode) prima dell'esecuzione.

- C
- Java (compiled to bytecode)

Virtual machines

Una VM è un'implementazione software di una macchina astratta (un computer "fittizio" con le sue istruzioni). È l'ambiente in cui viene eseguito il codice intermedio (come il bytecode di Java).

- Java bytecode runs on an interpreter
- interpreter often aided by a JIT compiler

Il codice è pre-compilato in un bytecode (codice intermedio).
La VM agisce da interprete per questo bytecode.

Traduce le righe di codice da ambiente virtuale per il processore fisico

Le prime tre fasi definiscono il Front-End del Compilatore, mentre le ultime due il Back-end

The Structure of a Compiler

(Il Compilatore é un traduttore: se il check del front-end é corretto, si esegue ottimizzazione e traduzione; se non é corretto, il compilatore si ferma e non continua con la traduzione)

Analogia Umana: Riconoscimento delle parole.

Analisi Lessicale

1. Scanning (Lexical Analysis)

Lo scanner legge il codice sorgente come una sequenza di caratteri e li raggruppa in unità significative chiamate token.

Analisi Sintattica

2. Parsing (Syntactic Analysis)

Il parser prende il flusso di token generato dallo Scanner e verifica se la sequenza di Token rispetta la grammatica (le regole sintattiche) del linguaggio di programmazione.

Analisi Semantica

3. Type checking (Semantic Analysis)

Il compilatore verifica se le strutture sintatticamente corrette hanno anche senso (significato) dal punto di vista del linguaggio (es. compatibilità dei tipi, variabili dichiarate).

4. Optimization

Ottimizzazione (miglioramento) del codice intermedio, senza alterarne la funzionalità.

5. Code Generation

Traduzione del codice intermedio ottimizzato in codice macchina specifico per l'architettura target (CPU).

Analogia Umana:
Riconoscimento della struttura grammaticale della frase.

Analogia Umana:
Comprensione del significato.

The first 3, at least, can be understood by analogy to how humans comprehend English.

1

Lexical Analysis

Divide il programma in token e controlla se sono corretti

- Lexical analyzer divides program text into “words” or “tokens”

if x == y then z = 1; else z = 2;

- Units:

if, x, ==, y, then, z, =, 1, :, else, z, =, 2, ;

2

Parsing

Il Parser prende la sequenza di token e verifica se rispetta le regole del linguaggio, costruendo una struttura gerarchica chiamata AST

- Once words are understood, the next step is to understand sentence structure

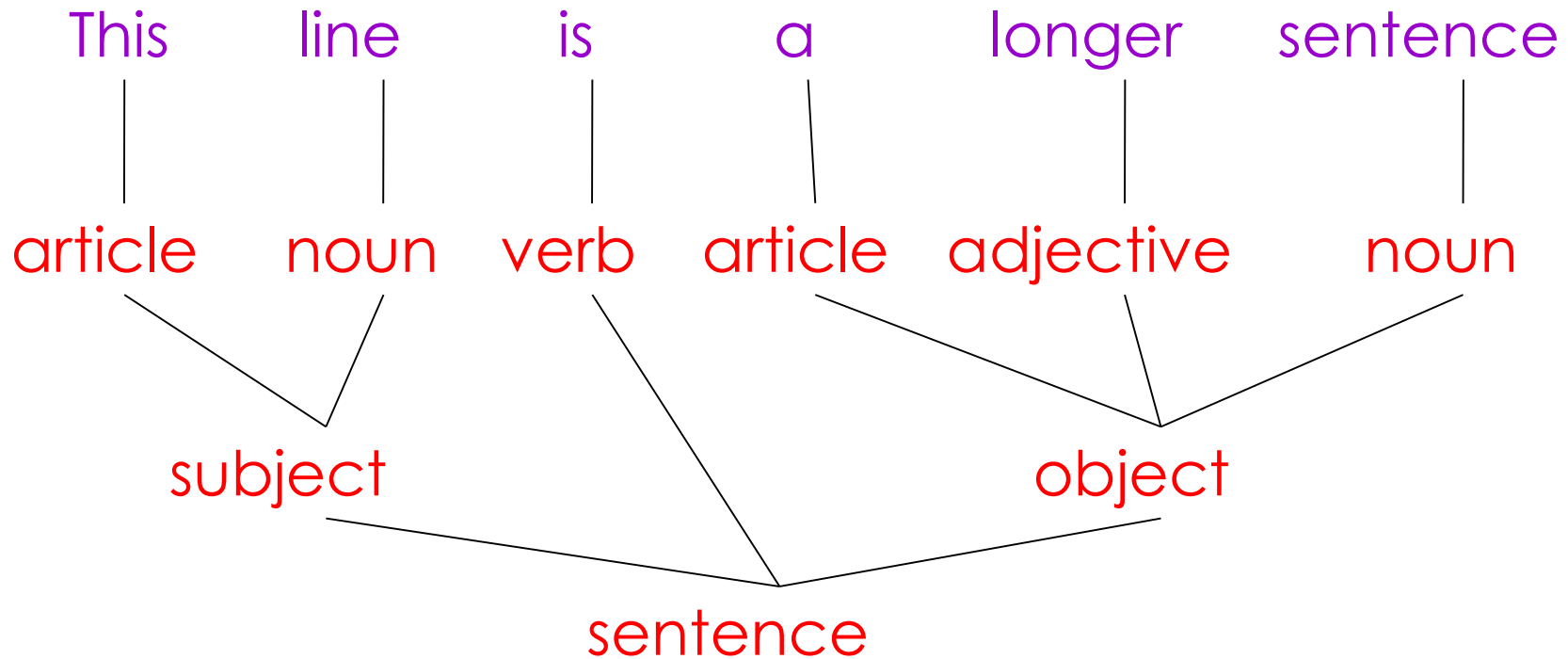
Strutturare le frasi

- Parsing = Diagramming Sentences
 - The diagram is a tree

Cercare la struttura nelle frasi del linguaggio parlato umano,
é uguale a farlo con il linguaggio di programmazione

Diagramming a Sentence

(Parser Bottom-Up, anche se viene mostrato al
contrario, cioè la radice si trova in basso)

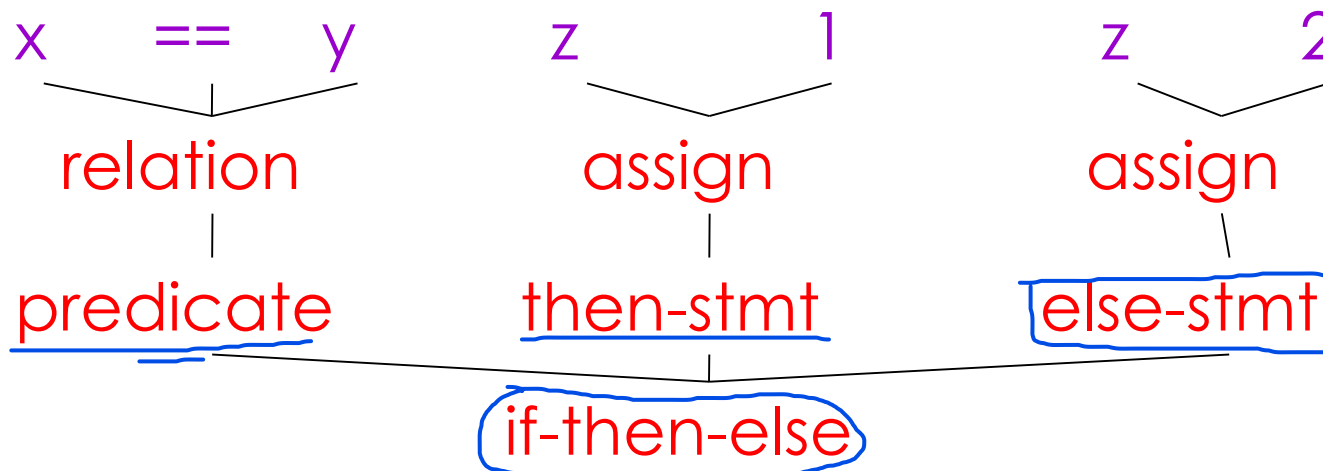


Parsing Programs

- Parsing program expressions is the same
- Consider:

If x == y then z = 1; else z = 2;

- Diagrammed:



3 Semantic Analysis in English

- Example:

Jack said Jerry left his assignment at home.

What does “his” refer to? Jack or Jerry?

- Even worse:

Jack said Jack left his assignment at home?

How many Jacks are there?

Which one left the assignment?

Semantic Analysis I

- Programming languages define strict rules to avoid such ambiguities

(that we have in English)

↳ Multiple defined name
or
Undefined name

- This C/C++ code prints "4"; the inner definition is used

↳ In Java, puoi dichiarare una sola volta una variabile
(quindi, in questo caso, avrebbe dato errore)

```
{  
    int Jack = 3;  
    {  
        int Jack = 4;  
        printf("%d", Jack);  
    }  
}
```

Semantic Analysis II

- Compilers also perform checks to find bugs
- Example:
Jack left her homework at home.
- A “type mismatch” between her and Jack
 - we know they are different people
(presumably Jack is male)

Code Generation

- A translation into another language
 - Analogous to human translation

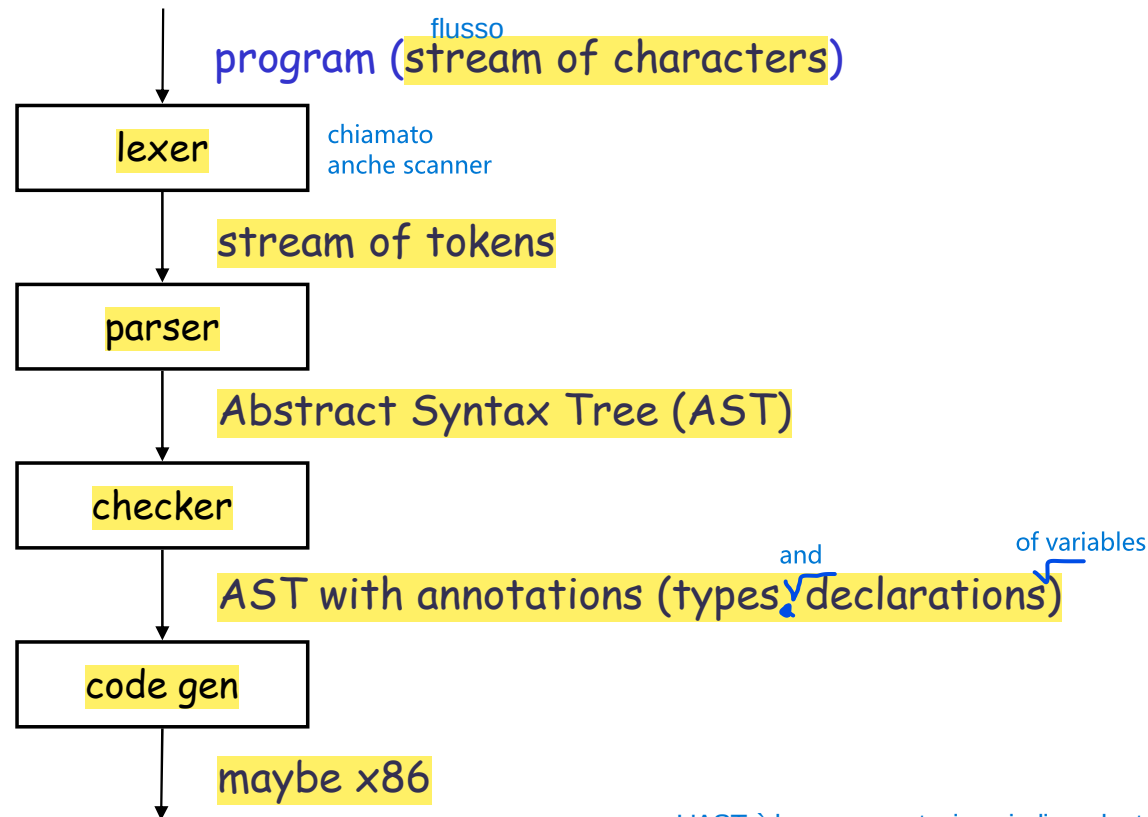
- Compilers for C, C++, ...
 - produce assembly code (typically)

- (e.g. GUI) code generators
 - produce C, C++, Java, ...

- Assembly Code: linguaggio a basso livello specifico per l'architettura della CPU (ancora rappresentazione testuale)
- Codice Oggetto: traduzione finale del codice sorgente (o dell'assembly) in una rappresentazione binaria che il processore può eseguire direttamente.

Quindi, L'Assembly è la versione testuale, simbolica e leggibile dall'uomo; il Codice Oggetto è la versione binaria, numerica e leggibile dalla macchina.

Summary: The Structure of a Compiler



L'AST è la rappresentazione indipendente dalla destinazione del codice sorgente, mentre la Generazione del Codice è la fase dipendente dalla destinazione ed è strutturata come un modulo intercambiabile (plug-in) per garantire flessibilità e portabilità.

Building a lexer

Recall: Lexical Analysis

Il Lexer viene utilizzato dal Parser (che viene chiamato come una funzione), che si occupa dell'analisi sintattica

- The input is just a sequence of characters. Example:

```
if (i == j)
    z = 0;
else
    z = 1;
```

- More accurately, the input is string:

```
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;
```

Individuare i Lessemi e mapparli sui Token

Goal: find lexemes and map them to tokens:

1. partition input string into substrings (called lexemes), and
2. classify them according to their role (role = token)

I Lessemi sono sottostringhe che matchano con il lessico del linguaggio. Ai lessemi vengono attribuiti dei ruoli (token)

Continued

- Lexer input:

`\tif(i==j)\n\t\tz = 0;\n\telse\n\t\tz = 1;`

- partitioned into these lexemes:

\	t	i	f	(	i	=	j	)	\	n	\	t	\	t	z		=		0	;	\	n	\	t	e	l	s	e	\	n	\	t	\	t	z		=		1	;
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	---	--	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	---	--	---	---

- mapped to a sequence of tokens

`IF, LPAR, ID("i"), EQUALS, ID("j") ...`

- **Notes:**

- whitespace lexemes are dropped, not mapped to tokens
- some tokens have **attributes**: the **lexeme** and/or **line number**

- **why do we need them?** Abbiamo bisogno degli attributi per due motivi principali: l'Analisi Semantica e il Debug.

1: Gli attributi contengono l'informazione specifica che trasforma un tipo di token astratto in un'entità concreta che può essere analizzata, memorizzata e, infine, tradotta in codice macchina.

2: Includendo il numero di riga come attributo, il compilatore può generare messaggi chiari e utili: "Errore sintattico: Punto e virgola mancante alla riga 5."

How to build a lexer?

Writing the lexer

- Not by hand
 - ^{noioso} tedious, repetitious, error-prone, non-maintainable
- Lexer generator!
 - once we have the generator, we'll only describe the lexemes and their tokens ...
 - that is, provide language's lexical specification (the What)
 - ... and generate code that performs the partitioning
 - generated code hides repeated code (the How)
 - codice sempre uguale, indipendente da linguaggio di programmazione

Code generator: key benefit

- The lexer generator allows the programmer to focus on:
 - What the lexer should do,
 - rather than How it should be done.
- what: declarative programming
- how: imperative programming

Imperative lexer (in Java)

- Let's first build the lexer in Java, by hand:
 - to see how it is done, and where's the repetitious code that we want to hide e soprattutto mostrare perché é noioso e difficile
- A simple lexer will do. Only four tokens:

TOKEN	Lexeme
ID	a sequence of one or more letters or digits starting with a letter
EQUALS	"=="
PLUS	"+"
TIMES	"*"

Imperative lexer

```
c=nextChar();
if (c == '=' ) { c=nextChar(); if (c == '=' ) {return EQUALS;}}
if (c == '+' ) { return PLUS; }
if (c == '*' ) { return TIMES; }
if (c is a letter) {
    c=NextChar();
    while (c is a letter or digit) { c=NextChar(); }
    undoNextChar(c);
    return ID;
}
```

Maximal match rule

- Il lexer legge il carattere extra in Look-Ahead.
- Se il carattere extra non fa parte del lessema corrente, il lexer conferma il lessema.
- Per non perdere quel carattere (che appartiene al prossimo lessema o serve da separatore), l'operazione `undoNextChar()` riporta indietro l'indicatore del flusso di caratteri di una posizione.

- What is the need for `undoNextChar()`?

Funzionamento: Il lexer deve leggere un carattere extra per determinare se il lessema corrente è terminato.

it performs look-ahead, to determine whether the ID lexeme can be grown further

Per implementare la Regola del Match Massimale, il lexer deve spesso guardare oltre la fine effettiva del lessema. Questa è l'operazione di Look-Ahead.

- This is an example of maximal match rule:

Ogni volta che il lexer incontra una sequenza di caratteri che può formare un lessema valido, deve continuare a leggere il più a lungo possibile per formare il lessema più lungo e valido.

- this rule followed by all lexers
- the rule: the input character stream is partitioned into lexemes that are as large as possible

Ex.: in Java, “iffy” is not partitioned into “if” (the IF keyword) and “fy” (ID), but into “iffy” (ID)

Match Non Massimale: Se si fermasse a if, l'input iffy sarebbe scomposto in IF (token parola chiave) e poi fy (token ID), un errore grave.

Match Massimale (Corretto): Il lexer continua fino a che non può più crescere (fino a quando il carattere successivo non invalida il lessema o non esiste). In questo caso, iffy è il lessema valido più lungo e viene tokenizzato come un singolo ID("iffy").

Imperative Lexer: what vs. how

```
c=nextChar();
if (c == '=' ) { c=nextChar(); if (c == '=' ) {return EQUALS;}}
if (c == '+' ) { return PLUS; }
if (c == '*' ) { return TIMES; }
if (c is a letter) {
    c=NextChar();
    while (c is a letter or digit) { c=NextChar(); }
    undoNextChar(c);
    return ID;
}
```

- little logic....

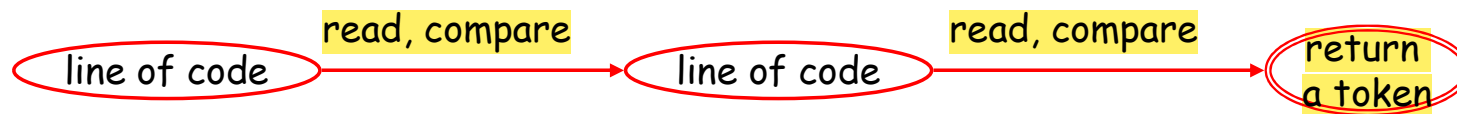
Separate out the **what** tramite DFA

```
c=nextChar();
if (c == '=') { c=nextChar(); if (c == '=') {return EQUALS;}}
if (c == '+') { return PLUS; }
if (c == '*') { return TIMES; }
if (c is a letter) {
  c=NextChar();
  while (c is a letter or digit) { c=NextChar(); }
  undoNextChar(c);
  return ID;
}
```

The diagram illustrates the control flow of the provided code. Red dots mark the start of the function, the entry into the first if-statement, the entry into the letter-handling block, and the entry into the while loop. Red arrows show the flow from these points to the corresponding return statements: EQUALS, PLUS, TIMES, and ID. The return ID path also includes a loop back to the while condition and an arrow to the undoNextChar(c) statement.

Separate out the **what**

- Is there a computational model that follows this behaviour?
 - yes, finite automata!



- The code actually follows this simple pattern:
 - read next character and compare it with some predetermined character
 - if there is a match, jump to a different line of code
 - repeat this until you return a token.

Legge prossimo carattere ed in base a quello decido cosa fare e nel caso ritorna il risultato finale (token). Dopo avere ritornato il token, ritorno nello stato iniziale del FA

A declarative lexer

Part 1: declarative (the what)

- describe each token as a finite automaton (or, better, as a regular expression)
 - must be supplied for each lexeme, of course (it specifies the lexical properties of the input language)

→ Oltre alla combinazione dei vari FA (unione dei FA ad uno stato iniziale), vengono aggiunti dei comandi negli stati di accettazione di ogni FA

Part 2: imperative (the how)

connect these automata into a lexer automaton

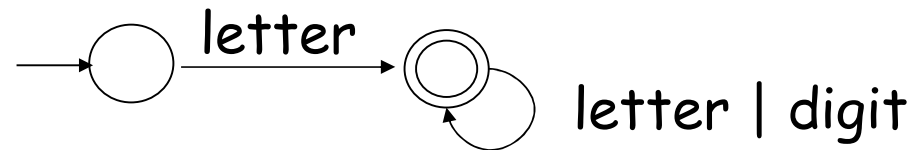
- common to all lexers (like a library)
- responsible for the mechanics of scanning

→ Prende tutti gli automi (i FA) generati dalle RE e li combina in un unico Automa Globale che riconosce simultaneamente tutti i possibili token.

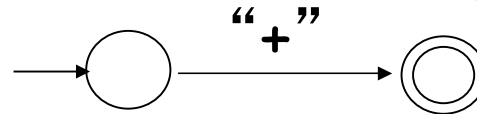
the declarative lexer

Part 1: specify a FA for each token

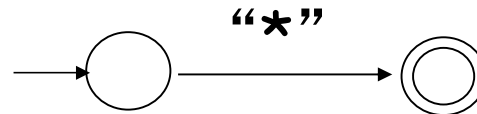
ID:



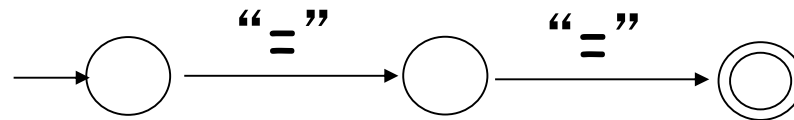
PLUS:



TIMES:



EQUALS:

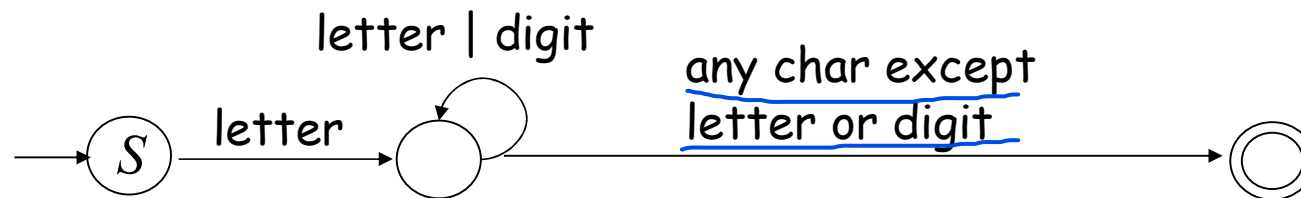


Part 2: add actions on acceptance of the FA

- the action can be one of
 - "put back one character" (if look-ahead is needed)
 - "return token XYZ"

Part 2: example of extending a FA

- The DFA recognizing identifiers is modified to:



action:

- put back 1 char
- return ID

→ Vedremo più avanti che non si applica solo ai lessemi con lunghezza variabile

Look-ahead is added for lexemes of variable length

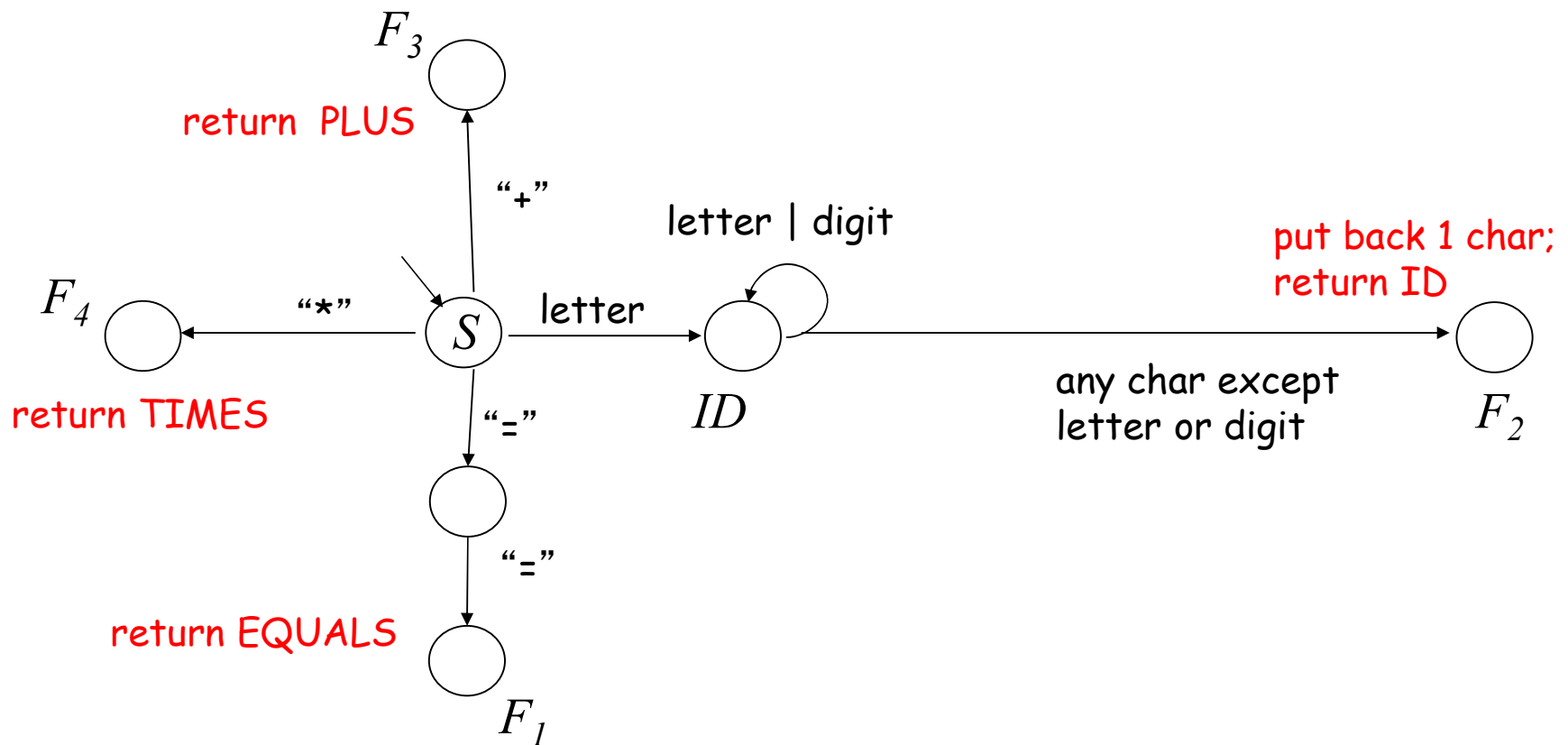
- in our case, only ID needs lookahead

- A note on action “**return ID**”

- resets the lexer back into start state S (ricorda che recall that lexer is called by parser; each time, one token is returned)

Part 2: Combine the extended FA's

The algorithm: merge start states of the DFA's.

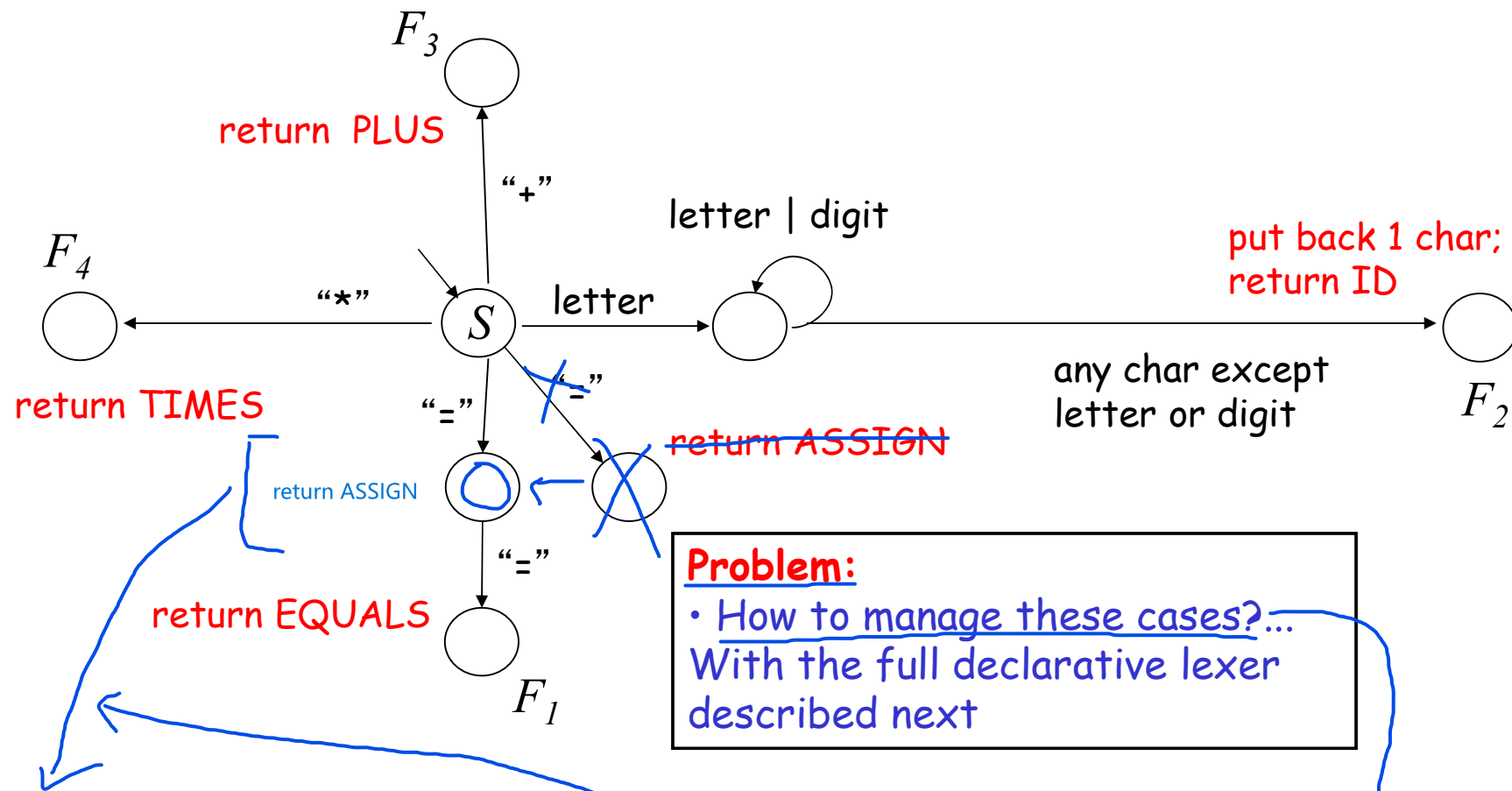


Towards a realistic scanner

- Consider a fifth token type, for the assignment operator

TOKEN	Lexeme
ID	a sequence of one or more letters or digits starting with a letter
EQUALS	"=="
ASSIGN	"="
PLUS	"+"
TIMES	"*"

But above algorithm produces



32

Inoltre, dobbiamo rendere il DFA da non deterministico a deterministico. Rendendolo deterministico però appunto dovremmo aggiungere anche il Maximal Match

In questo caso, se il prossimo lessema sarebbe $==$, in realtà il DFA vede un solo $=$ e lo associa al token ASSIGN. Come risolvere? Maximal Match. Ma quindi il Maximal Match non vale solo per i lessemi di lunghezza variabile.

La soluzione del Maximal Match viene aggiunta dal Full Declarative Lexer (il problema rimane nel Declarative Lexer che non applica Maximal Match quando trova il primo stato di accettazione)

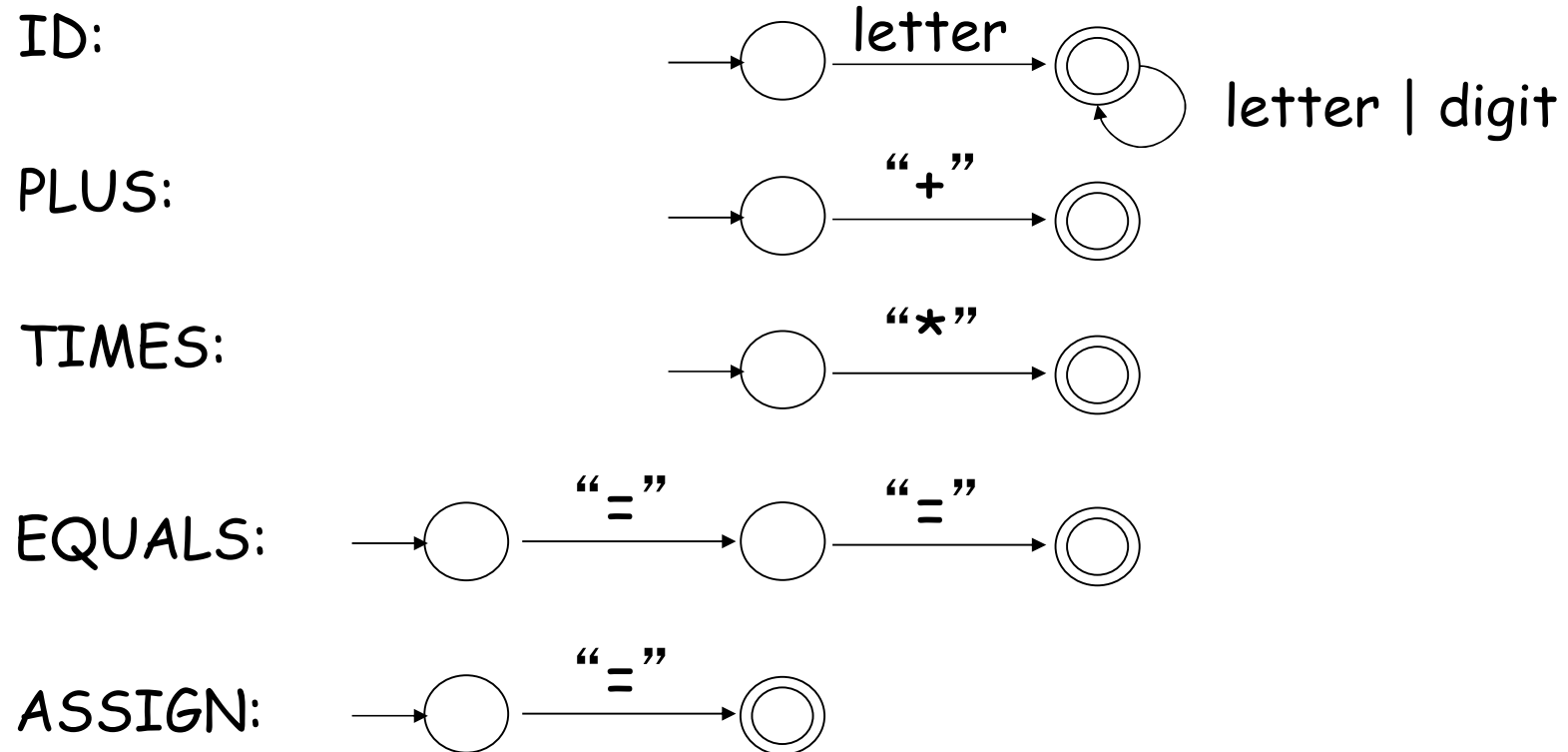
full declarative lexer

The algorithm

We can develop the lexer as follows:

1. Specify an automaton for each lexeme (as before)

Step 1 in our example



The algorithm

L'utente non si occupa dell'estensione (é l'algoritmo che applica maximal match)

We can develop the lexer as follows:

1. Specify an automaton for each lexeme ^{token} (as before)
2. Merge them (start states) into the lexer automaton
3. If the obtained automaton is nondeterministic, translate into a deterministic automaton
4. Consider the following rules (implementing maximal match):
 - whenever you reach a final state: (quindi, non si ferma più agli stati di accettazione)
 - remember position in input (so that you can undo reads) (ricordare la posizione dell'input nello stream degli input)
 - keep reading more characters, moving to other states
 - whenever you get stuck (cannot make a move on next char):
 - return to the last final state (i.e., undo the reads)
 - return the token associated with this final state

Notes

(Viene applicato il look-ahead fino a che l'algoritmo si blocca)

- Notice that reading past final state implements look-ahead
- This look-ahead is unbounded
 - can undo any number of characters

Practical concerns

- **Ambiguity**

- **problem:** lexer may reach multiple final states at once
- **ex.:** "if" matches both ID and IF (the keyword)
- **solution:** prioritize tokens (IF wins over ID)

dare ordine di priorità ai token

le Parole
Chiave hanno
una priorità
maggiore
rispetto agli
Identificatori.

- **Discarding whitespace**

- **solution:** final state for white-space lexeme is special: **don't return a token**, just jump to (common) start state

(es: spazi, tabulazioni, newline)

alla fine del lessema degli spazi vuoti,
non ritornare nulla, ma salta e ritorna
allo stato iniziale del DFA combinato

- **Error inputs**

- **problem:** discard illegal lexemes and print an error message
- **simple solution** (discard char by char): add a lexeme that matches any character, giving it lowest priority; it will match when no other

will

(e quel carattere sarà
scartato e dato errore)

Si definisce una regola lessicale "catch-all" (cattura-tutto) che corrisponde a qualsiasi singolo carattere (es. con la RegEx ., dot). Questa regola riceve la priorità più bassa di tutte. Se l'automa non riesce a far corrispondere l'input a un lessema valido (ID, IF, NUM, ecc.), cadrà per forza su questa regola catch-all. L'azione associata a questa regola è: stampare un messaggio di errore e scartare il carattere, per poi ripartire dallo stato iniziale.

Aggiungi una regola nel
tuo Lexer che accetti
come lessema un singolo
carattere qualsiasi

We have a full declarative scanner

- imperative part,
 - The steps 2, 3, and 4 of the algorithm
- declarative part
 - Specify each token with an automaton

But how to specify the automata in practice?

regular expressions

Regular Expressions

- Automaton is a good “visual” ^{aiuto} aid
 - but is not suitable as a specification
(its textual description is too clumsy)
- regular expressions are a suitable *specification*
 - a compact way to define a language that can be accepted by an automaton.
- used as the input to a lexer generator
 - define each token, and also
 - define white-space, comments, etc
 - these do not correspond to tokens,
but must be recognized and ignored.

Example: identifier

- Lexical specification (in English):
 - a letter, followed by zero or more letters or digits.
- Lexical specification (as a regular expression):
 - $[a-zA-Z]([a-zA-Z] | [0-9])^*$

$[a-zA-Z]$ concatenato ($[a-zA-Z]$ oppure $[0-9]$) stella di Kleene

	means "or"	(nelle RE teoriche, sarebbe il '+')
.	means "followed by"	concatenazione
*	means zero or more instances of	stella di Kleene
()	are used <u>for grouping</u>	per dare precedenza

Operands of a regular expression

- Operands are same as labels on the edges of an FA
 - single characters, or
 - the special empty string character ϵ
(in ANTLR denoted by not writing any character!)
- sometimes we put the characters in quotes, e.g. 'a' (required in ANTLR)
 - necessary when denoting | . *
- [a-z] (written 'a'..'z' in ANTLR) is a shorthand for
 - a | b | c | ... | z
- [0-9] (written '0'..'9' in ANTLR) is a shorthand for
 - 0 | 1 | ... | 9

| . * operators have implicit precedence!

Regular Expression Operator	Analogous Arithmetic Operator	Precedence
	plus	lowest
.	times	middle
*	exponentiation	highest

- Consider regular expressions:
 - $[a-zA-Z] \cdot [a-zA-Z] \mid [0-9]^*$
 - $[a-zA-Z] \cdot ([a-zA-Z] \mid [0-9])^*$

Example: Integer Literals

- An integer literal with an optional sign can be defined in English as:
 - “(nothing or + or -) followed by one or more digits”
- The corresponding regular expression is:
 - $(+|-|\epsilon).[0-9].[0-9]^*$
- "." can also be omitted, e.g.
 - $(+|-|\epsilon) [0-9] [0-9]^*$
- A new convenient operator '+'
 - same precedence as '*'
 - $[0-9].[0-9]^*$ is the same as
 - $[0-9]^+$ which means "one or more digits"

Lexer generators

Lexer generators

L'utente manda in input al Generatore di Lexer le RE.
Il Generatore trasforma le RE in FA.

- Receive in input the regular expressions describing the lexemes
- Generate code (C, C++, Java, ...) that implements the full declarative lexer algorithm:
 - Translate the regular expressions into FA
 - Merge the FA in a unique automaton
 - Translate the merged automaton to a Deterministic FA (more efficient to be simulated)
 - Produce the code that implements the "special" simulation of the DFA (lookahead for maximal match rule, priorities in case of multiple match, operations to be executed upon matching, and return to initial state)

Implementation

Invece di scrivere codice imperativo con molti if/else o switch per ogni stato e per ogni carattere, il DFA è rappresentato da una struttura dati molto efficiente: una matrice bidimensionale (Tabella T)

- A DFA can be implemented by a 2D table T

- One dimension is “states”

Le righe corrispondono agli stati interni del DFA ($S_0, S_1, \dots, S_n$).

- Other dimension is “input symbols”

Le colonne corrispondono a tutti i possibili caratteri di input (l'alfabeto, es. lettere, cifre, simboli).

- For every transition $S_i \xrightarrow{a} S_k$ define $T[i,a] = k$

→ In ogni cella della tabella $T[i, a]$, è memorizzato l'indice dello stato successivo (S_k) che deve essere raggiunto.

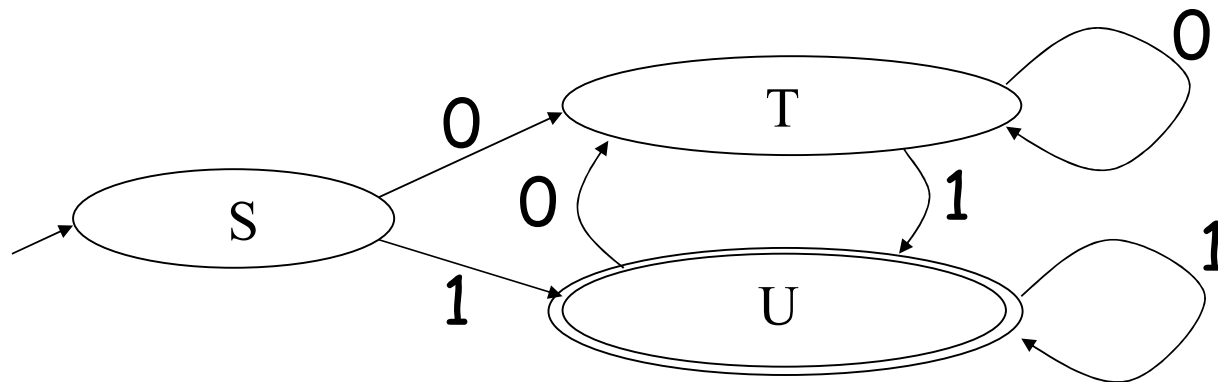
- DFA “execution”

- If in state S_i and input a , read $T[i,a] = k$ and skip to state S_k

- Very efficient

→ L'esecuzione del lexer si riduce a una serie di semplici e veloci operazioni di ricerca in tabella (lookup) e cambio di stato.

Table Implementation of a DFA



simboli dell'alfabeto

	0	1
S	T	U
T	T	U
U	T	U

stati interni del DFA

Lexer generator per parser bottom-up

Example: **flex (a fast lexer generator)**

- **flex generates lexers**
- **flex input:**
 - file with regular expressions defining the tokens, and for each token some **C** instructions
- **flex output:**
 - a **C** file named **lex.yy.c** that contains the definition of a function **yylex()**
 - **yylex()** reads the input and finds the tokens
 - when a token is found the corresponding **C** instructions are executed

Example of flex input file

Dall'alto verso il basso, il lessema con più priorità

```
%{  
#include <stdio.h>  
%}  
%%  
[ \t\n]+    printf("white space, length %i\n", yyleng);  
"*"        printf("times\n");    "/"        printf("div\n");  
"+"        printf("plus\n");    "-"        printf("minus\n");  
"("        printf("left-par\n");  ")"        printf("right-par\n");  
0|([1-9][0-9]*)    printf("integer %s\n", yytext);  
[a-zA-Z_][a-zA-Z0-9_]*    printf("identifier %s\n", yytext);  
.  
%%  
main(){ yylex(); }
```

lunghezza del lessema

lessema matchato

funzione che lancia la funzione del lexer

Building a parser

Il Parse Tree è la prima rappresentazione generata dal parser (Analizzatore Sintattico) e riflette l'intera struttura sintattica del codice sorgente, basandosi direttamente sulla Grammatica Context-Free (CFG) del linguaggio.

- Scopo Principale (Sintassi Corretta): Eseguire il Controllo Sintattico (Syntax Checking). Dimostra che la sequenza di token fornita dal scanner può essere derivata dalle regole della grammatica.

- Contenuto: Contiene tutti gli elementi della derivazione: I nodi interni sono simboli non terminali (le variabili grammaticali); I nodi foglia sono i token terminali.

- Dettagli Inclusi: Mantiene tutti i simboli della grammatica ma anche quelli irrilevanti per il significato logico del programma (es. parentesi, punti e virgola).

- Ruolo per il Parser: Il parser usa l'esistenza implicita o esplicita del Parse Tree per definire le relazioni di annidamento (nesting) tra le espressioni (sotto-alberi e sopra-alberi).

Overview

- What does a parser do, again?

its two tasks

Albero Sintattico (teoria)

- parse tree vs. AST

- A hand-written parser

- and why it gets hard to get it right

Controlla la sequenza di token
dati dal lexer/scanner e forma
un programma corretto

Genera un albero sintattico (top-down o bottom-up)

L'AST è la rappresentazione finale, ottimizzata per le fasi successive di analisi e traduzione. Viene costruito trasformando o filtrando il Parse Tree.

- Scopo Principale (Sintassi Astratta): Fornire una rappresentazione pulita e essenziale del programma, contenente solo ciò che è necessario per l'Analisi Semantica, l'Ottimizzazione e la Generazione del Codice.

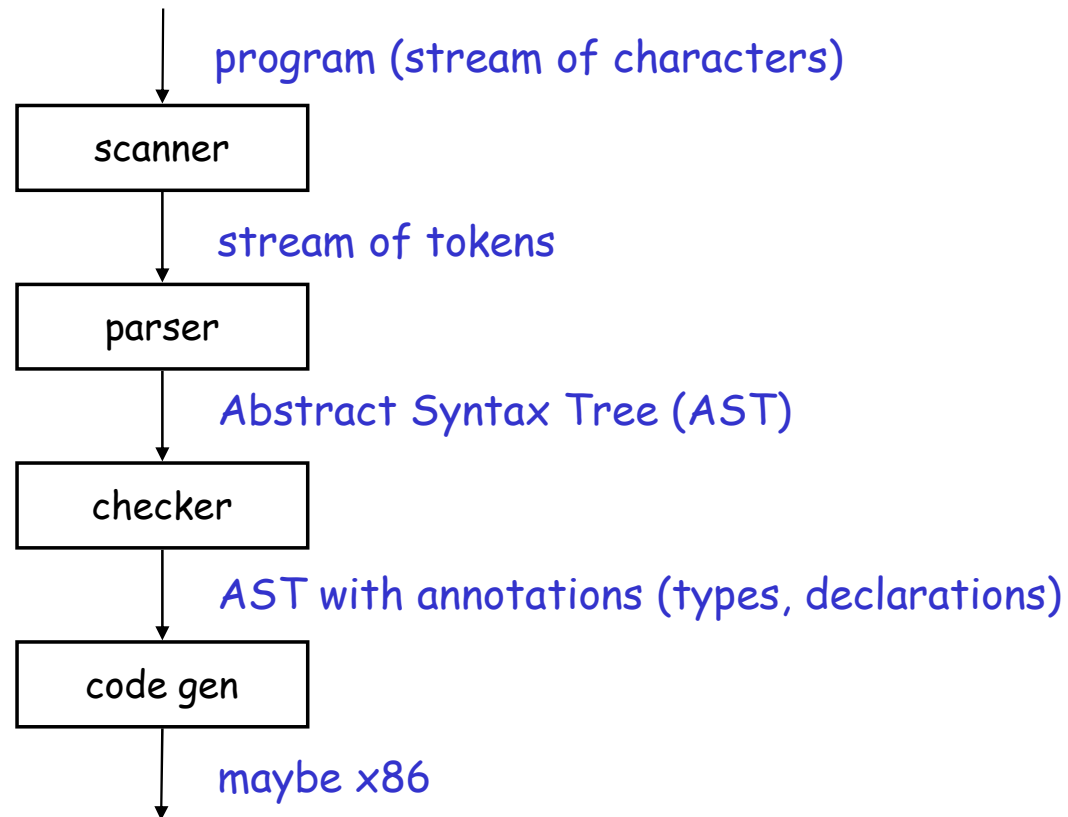
- Contenuto: I nodi interni rappresentano gli operatori o le costruzioni logiche (es. assegnazione, somma, ciclo while); I nodi foglia rappresentano gli operandi o i valori (es. variabili/identificatori, costanti).

- Dettagli Esclusi: L'AST rimuove tutto ciò che è superfluo ai fini della logica del programma, come parentesi, punti e virgola e simboli non terminali intermedi.

- Flusso: Il parser genera il Parse Tree (implicitamente o esplicitamente) prima di generare l'AST. La costruzione dell'AST è il risultato finale dell'analisi sintattica.

What does a parser do?

Recall: The Structure of a Compiler



Recall: Syntactic Analysis

Obiettivo: Verificare se sequenza di input da scanner rispetta la Grammatica del linguaggio e, in caso affermativo, rappresentarne la struttura gerarchica.

- **Input:** sequence of tokens from scanner
- **Output:** abstract syntax tree (AST)
- **Actually,**
 - parser first considers the parse tree, i.e. the (non abstract) syntax tree
 - AST is then built by translating the parse tree
 - parse tree rarely built explicitly; only determined by, say, how parser pushes stuff to stack
 - our lectures first focus on constructing the parse tree; later we'll show the translation to AST.

Example

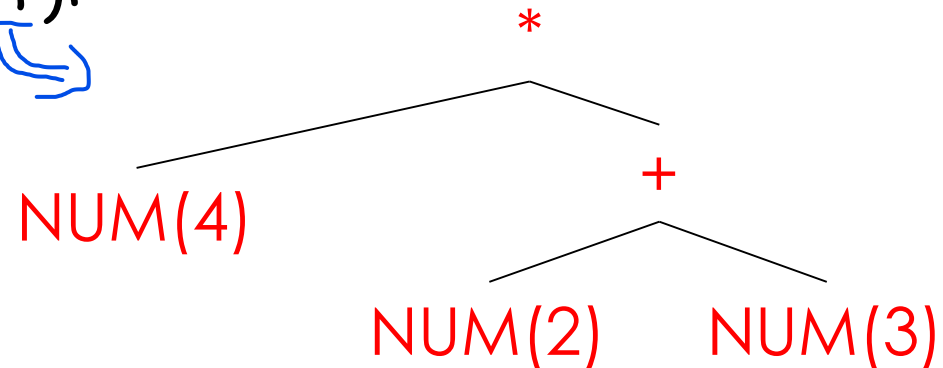
- Expression

$4*(2+3)$

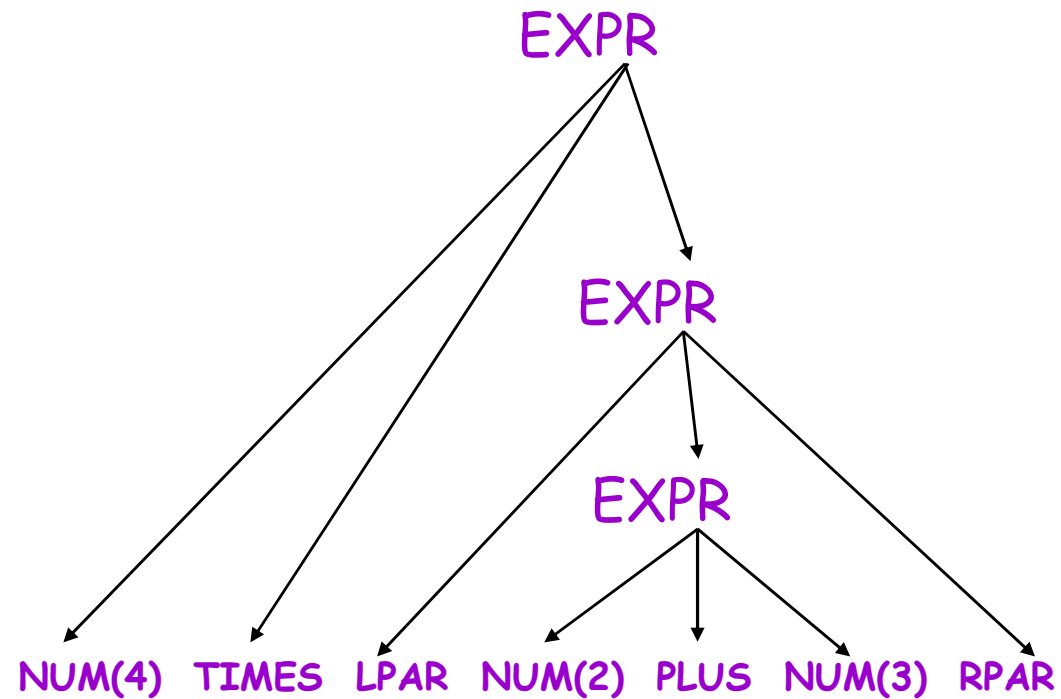
- Parser input sequenza di token individuati dallo scanner/lexer

NUM(4) TIMES LPAR NUM(2) PLUS NUM(3) RPAR

- Parser output (AST):



Parse tree for the example



leaves are tokens

Another example

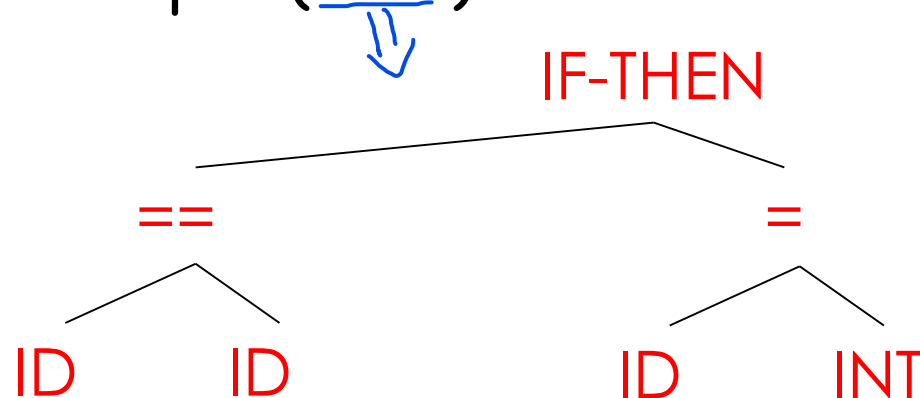
- Other expression

if (x == y) { a=1; }

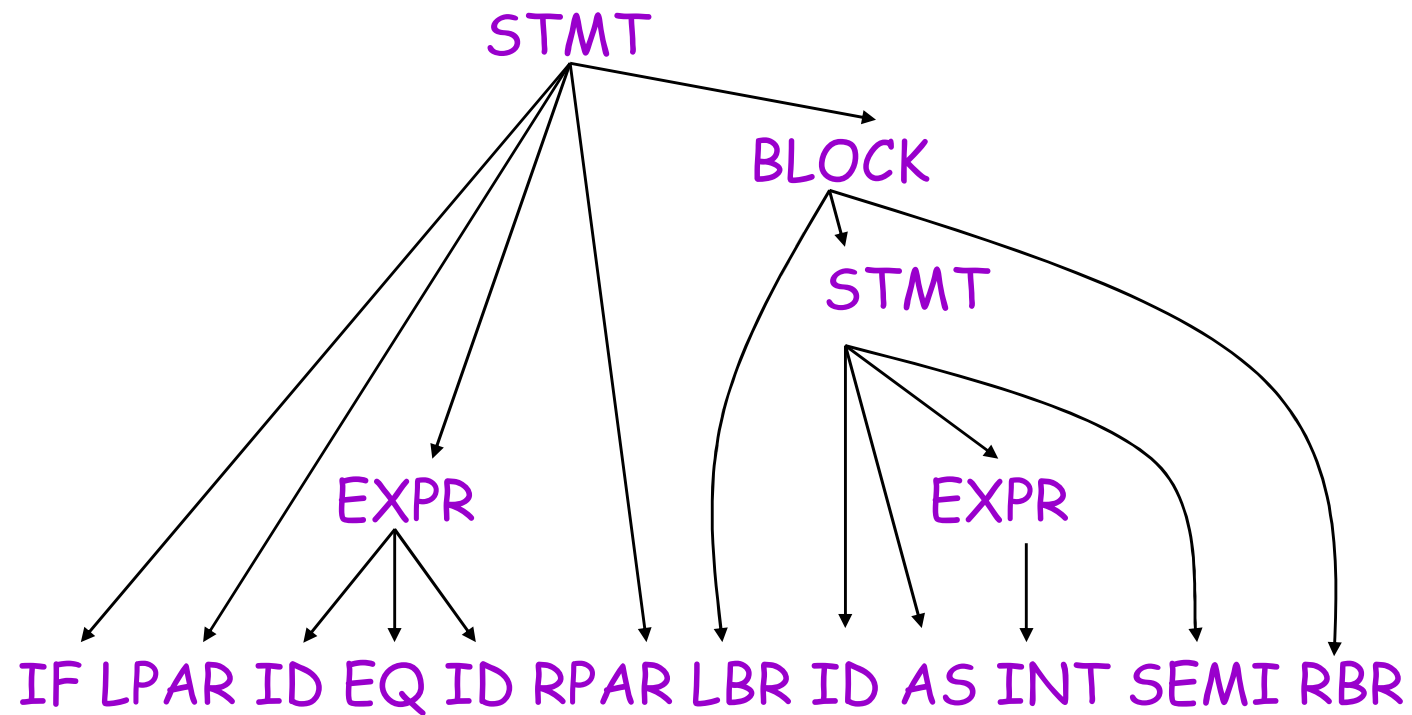
- Parser input

IF LPAR ID EQ ID RPAR LBR ID AS INT SEMI RBR

- Parser output (AST):



Parse tree for the example



leaves are tokens

Parse tree vs. abstract syntax tree

- **Parse tree** Il Parse Tree è la rappresentazione letterale della grammatica. Mostra ogni singolo passaggio che il parser ha fatto per validare la stringa.
 - contains all tokens, including those that parser needs per individuare
“only” to discover
 - ~~intended~~ annidamento nesting: parentheses, curly braces
 - terminazione di istruzione statement termination: semicolons
 - technically, parse tree shows **concrete syntax**
- **Abstract syntax tree (AST)** L'AST è la versione "ripulita" e ottimizzata. Rappresenta la gerarchia logica del codice, eliminando ciò che non serve più alle fasi successive.
astrae gli artefatti dell'analisi sintattica
 - abstracts away artifacts of parsing, by flattening tree hierarchies, dropping tokens, etc.
 - technically, AST shows **abstract syntax**

Comparison with Lexical Analysis

<i>Phase</i>	<i><u>Input</u></i>	<i><u>Output</u></i>
<u>Lexer</u>	Sequence of characters	Sequence of tokens
<u>Parser</u>	Sequence of tokens	AST, built from parse tree

Summary

Il Parse Tree viene costruito nella fase 1 per Syntax Checking e viene utilizzato nella fase 2 per la generazione dell'AST

- Parser performs two tasks:

- 1 - Syntax checking

- a program with a syntax error is rejected and information about error given

- 2 - Parse tree construction

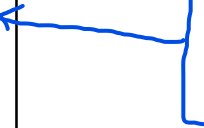
- usually implicit (needed for syntax checking)
- exploited to build the AST

How to build a parser?

Writing the parser

- Can do it all by hand, of course
 - ok for small languages, but hard for real programming languages
- Just like with the scanner, it is possible to use a parser generator
 - one concisely describes the syntactic structure
 - that is, how expressions, statements, definitions look like
 - and the generator produces a working parser
- Let's start with a hand-written parser
 - to see why we want a parser generator

Definiamo la
grammatica
del linguaggio
e ci genera il
Parser



First example: balanced parens

- Our problem: check the syntax
 - are parentheses in input string balanced?
- The simple language
 - parenthesized number literals (numeri letterali tra parentesi)
 - Ex.: 3, (4), ((1)), (((2))), etc
- Before we look at the parser
 - why aren't finite automata sufficient for this task?

Gli automi a stati finiti (FA) non hanno memoria. O meglio, hanno una memoria estremamente limitata che non permette loro di gestire strutture "annidate" o bilanciate.

L'Automa a Pila è ciò che ci serve per il parsing. È come un automa normale, ma con l'aggiunta di uno Stack

Parser code preliminaries

- Let TOKEN be an enumeration type of tokens:
 - NUM, LPAR, RPAR, PLUS, MINUS, TIMES, DIV
- Let the global in[] be the input string of tokens (Generato dal Lexer)
- Let the global next be an index in the token string (initially is 0)

Se facciamo il Parser By Hand, possiamo usare la Ricorsione insieme allo Stack

Parsers use stack to implement infinite state

Balanced parentheses parser:

Post-increment (i++): The original value of i is returned by the expression, and then i is incremented. The side effect is the change to the variable i itself.

```
void Parse() {  
    nextToken = in[next++];  
    if (nextToken == NUM) return;  
  
    if (nextToken != LPAR) print("syntax error");  
    Parse();  
    if (in[next++] != RPAR) print("syntax error");  
}
```

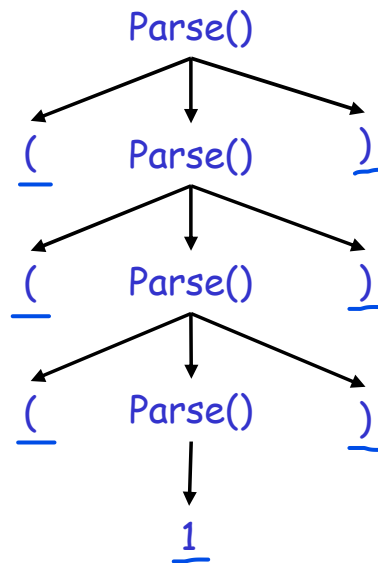
Il Parser non effettua controlli dopo la parentesi chiusa o non rileva errori in caso di sola parentesi chiusa (es: '5')

Leggo il prossimo Token, se é NUM ritorno; se non é un LPAR/RPAR, errore;
Tra LPAR e RPAR chiama Parse().

In pratica, quello che deve fare é verificare che le parentesi siano bilanciate
(se ho una parentesi aperte, ci deve essere la corrispondente chiusura)

Where's the parse tree constructed?

- In this parser, the parse is given by the call tree:
- For the input string ((1)) :



Leggo le foglie dall'alto a sinistra
verso il basso e poi ritorno dal
basso verso l'alto a destra

Second example: subtraction expressions

The language of this example:

1, 1-2, 1-2-3, (1-2)-3, (2-(3-4)), etc

```
void Parse() {  
    if ((nextToken = in[next++]) == NUM) {  
        if (in[next] == MINUS) { next++; Parse(); }  
    } else if (nextToken == LPAR) {  
        Parse();  
        if (in[next++] != RPAR) print("syntax error");  
    } else print("syntax error");  
}
```

Questa implemetazione
si blocca, se dopo la
parentesi c'è un MINUS

Leggo il prossimo Token;
- se NUM, guardo se il prossimo è un MINUS, se è un MINUS,
mando avanti l'indice sullo stream dei token e chiamo Parse;
- se LPAR, chiamo il Parse e se il prossimo token non è una
parentesi chiusa, ho errore.
- negli altri casi, ho errore

Second example: subtraction expressions

The language of this example: (fixed!!)

1, 1-2, 1-2-3, (1-2)-3, (2-(3-4)), etc

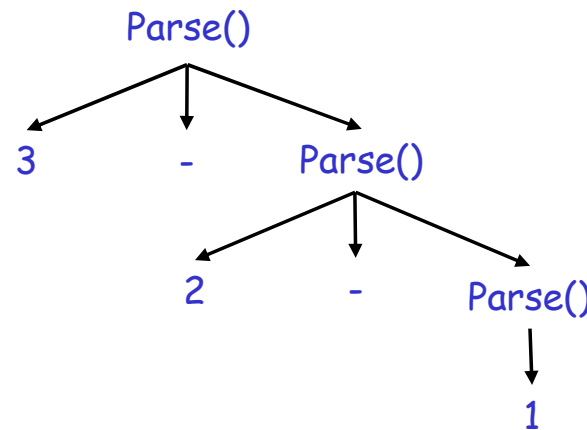
```
void Parse() {  
    if ((nextToken = in[next++]) == NUM) {  
        if (in[next] == MINUS) { next++; Parse(); }  
    } else if (nextToken == LPAR) {  
        Parse();  
        if (in[next++] != RPAR) print("syntax error");  
        if (in[next] == MINUS) { next++; Parse(); }  
    } else print("syntax error");  
}
```

Per risolvere il problema del "MINUS dopo parentesi" é stato aggiunto questo controllo che permette di avere il MINUS dopo la parentesi aperta

Subtraction expressions continued

- Observations:
 - a more complex language
 - hence, harder to see how the parser works (and if it works correctly at all)
 - the parse tree is actually not really what we want
 - consider input 3-2-1
 - what's undesirable about this parse tree's structure?

Con associatività a destra, il risultato finale é 2, mentre dovrebbe essere, tramite associatività a sinistra, 0



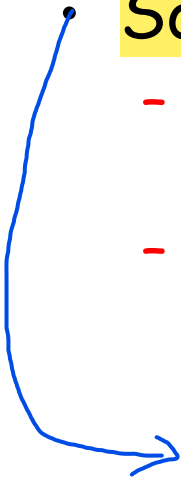
Il Parse Tree produce un prodotto che non é quello che ci aspettavamo (perché implementa associatività a destra dell'addizione)

We need a clean syntactic description

- Just like with the scanner, writing the parser by hand is painful and error-prone
 - consider adding +, *, / to the last example!

So, let's separate the what and the how

- **what:** the syntactic structure, described with a context-free grammar
- **how:** the parser, built from the grammar, which reads the input and produces the parse tree



Quindi, usiamo la grammatica CFG (What?) per generare il Parse Tree (How?)

Idea...

- We can describe the syntactic structure by using context-free grammars!
- What's next?
 - Grammars
 - Derivations

Grammars

La struttura del

-  Programming language constructs have recursive structure.
 - which is why our hand-written parser had this structure, too
- Example: an **expression** is either:
 - **number**, or
 - **variable**, or
 - **expression + expression**, or
 - **expression - expression**, or
 - **(expression)**, or
 - ...

Context-free grammars (CFG)

- a natural notation for this recursive structure
- grammar for our balanced parens expressions:
 $\text{BalancedExpression} \rightarrow a \mid (\text{BalancedExpression})$
- describes (generates) strings of symbols:
 - $a, (a), ((a)), (((a))), \dots$
- like regular expressions but can refer to
 - other expressions (here, $\text{BalancedExpression}$)
 - and do this recursively (thus, it is “non-finite state”)

Nelle RE, non ho variabili (quindi non posso usare la ricorsione). Inoltre, nelle CFG, riesco ad andare oltre il Final State (uso ricorsione)

Example: arithmetic expressions

- Simple arithmetic expressions:

$E \rightarrow n \mid id \mid (E) \mid E + E \mid E * E$

- Some strings of this language:

- id
- n
- (n)
- n + id
- id * (id + id)

Il problema è che le grammatiche generano le stringhe, cioè la CFG specifica le regole per passare dal simbolo iniziale alla stringa, mentre il Parsing richiede il Riconoscimento delle stringhe, cioè dobbiamo riconoscere se una derivazione esiste e costruire il parse tree per confermarne la validità.

derivazioni

How do derivations help us in parsing?

errori sintattici

- A program (a string of tokens) has no syntax error if it can be derived from the grammar. (cioè il programma è corretto, se è derivabile dalla grammatica)

grammars, however, define how to derive some (any) string, not how to check if a given string is derivable

- So how to do parsing?
 - a naive solution: derive all possible strings and check if your program is among them derivare tutte le possibili stringhe e controllare
 - not as bad as it sounds: there are parsers that do this! cioè l'idea alla base è quella ma ottimizzata
Coming soon.

I primi parser in assoluto, usavano la soluzione naive: cerco il programma tra tutti i programmi possibili

Example

A fragment of a simple language:

STMT → while (EXPR) STMT
| id (EXPR) ;

EXPR → EXPR + EXPR
| EXPR - EXPR
| EXPR < EXPR
| (EXPR)
| id

Un'espressione è un costrutto che viene valutato per produrre un singolo valore. Scopo: Calcolare un valore. Risultato: Restituisce sempre un valore.

Uno statement è un costrutto che viene eseguito per compiere un'azione (effettuare un cambiamento di stato, controllare il flusso, ecc.). Scopo: Eseguire un'azione o controllare il flusso del programma. Risultato: Non restituisce necessariamente un valore utilizzabile. Se un valore viene prodotto, di solito viene scartato o ignorato nel contesto dello statement.